プログラミングに
数学的「安心」を

室屋 晃子
(お茶の水女子大学 理学部情報科学科 講師)

理学総論・情報科学特別講義, 21 January 2026

室屋とは：新任教員@育児休業中

室屋 晃子（むろや こうこ / Koko Muroya）

- 学生として：東大情報科学科

  → 東大コンピュータ科学専攻 修士

  → 🇬🇧バーミンガム大学コンピュータ科学専攻 博士

- 研究者として：京大 → 国立情報学研究所 → お茶大（今ここ）

- 0歳5ヶ月👶のため育児休業中

  → 4月から復帰 & 研究室立ち上げ予定！

- プログラミングの「安心」を数学を使って保証したい

1. 表現と証明で「安心」を保証する

2. いつでも簡単に「安心」を保証する

（表現と証明の工夫）

3. 簡単から、いつでもへ

（表現と証明の工夫の実例）

4. AI時代にも「安心」を保証する

（「安心」は変化するが、やはり重要）

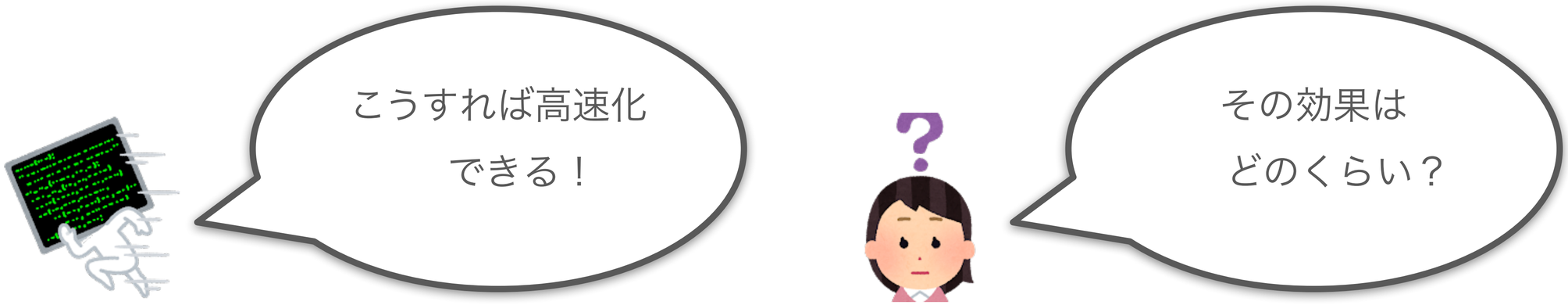
研究の目標：「安心」してプログラミングがしたい

- 正しさの安心：「ちゃんと（意図通りに）」プログラムが動作する

研究の目標：「安心」してプログラミングがしたい

- 速さの安心：「ちゃんと（適切に）」プログラムが**高速化**される

- 高速化：「プログラムをこう書いた方が速く動作する」
  - 人やAIが行ったり、コンパイラが自動で行ったりする
  - 向き不向きがあったり、組み合わせに難があったりする



「安心」のために：テストと保証

- Q. 「2つのプログラムが同じ動作をするが、片方はより速い」ことを確かめるには？

- A1. 実機でテストする
  - 出力と実行時間をデータで比較する

「安心」のために：テストと保証

- Q. 「2つのプログラムが同じ動作をするが、片方はより速い」ことを確かめるには？

- A2. 理論的に、数学を使って保証する
  - 出力の等しさと実行時間の差（の見積もり）を証明する
  - 手間はかかるが、より安心できる！
    - 個々のコンピュータ・環境に依らない
    - 全てのケースを網羅している

「安心」の保証：表現してから証明する

- 2つのプログラムの、出力の等しさと実行時間の差を**証明**する
- そのためには、まず数学的に**表現**する必要がある
  - プログラムの動作過程
  - プログラムの出力・実行時間

数学的な表現の例：「操作」の繰り返し

手続き型プログラム

```
①   x = 1;
②   y = 2;
③   return x + y;
```

動作過程の表現

```
   ①, []
→ ②, [(x,1)]
→ ③, [(x,1); (y,2)]
→ 3
```

操作：変数xと値1の対応を追加して、行番号を進める

操作：変数yと値2の対応を追加して、行番号を進める

操作：結果3を計算する

出力の表現は、最終結果「3」

実行時間の表現は、操作の回数「3」

数学的な表現の例：「操作」の繰り返し

関数型プログラム

```
(fun x -> x * x) 3;
```

動作過程の表現

```
   (fun x -> x * x) 3
→ 3 * 3
→ 9
```

操作：変数xに値3を代入する

操作：掛け算3*3を計算する

参考：数式の等式変形

```
f(x) = x * x のとき
f(3) = 3 * 3
     = 9
```

出力の表現は、最終結果「9」

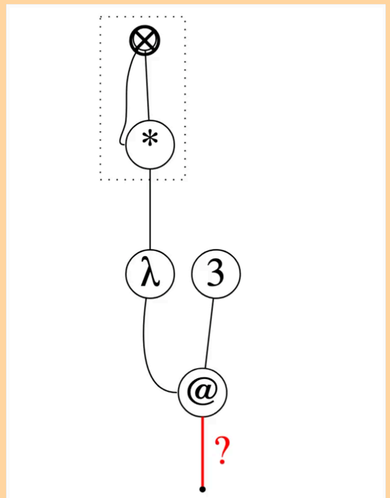
実行時間の表現は、操作の回数「2」

数学的な表現の例：「操作」の繰り返し

関数型プログラム

```
(fun x -> x * x) 3;
```

動作過程の表現



操作：構文木っぽいグラフを変形する

出力の表現は、最終結果「9」

実行時間の表現は、操作の回数

数学的な表現の例：「操作」の繰り返し

プログラムの〇〇	数学的な表現
動作過程	「操作」の繰り返し
出力	「操作」の最終結果
実行時間	「操作」の回数

動作過程の表現

```
  (fun x -> x * x) 3
→ 3 * 3
→ 9
```

操作：代入
操作：掛け算

出力の表現は、最終結果「9」

実行時間の表現は、操作の回数「2」

表現と証明で「安心」を保証する

- 数学的に**表現**する

プログラムの〇〇	数学的な表現
動作過程	「操作」の繰り返し
出力	「操作」の最終結果
実行時間	「操作」の回数

- 2つのプログラム P, Q の、出力の等しさと実行時間の差を**証明**する

命題　P の動作過程 $D_1 \to \cdots \to D_N \not\to$ に対して

Q の動作過程 $E_1 \to \cdots \to E_M \not\to$ は $D_N = E_M$ かつ $N \geq M$ を満たす

1. 表現と証明で「安心」を保証する

2. いつでも簡単に「安心」を保証する

（表現と証明の工夫）

3. 簡単から、いつでもへ

（表現と証明の工夫の実例）

4. AI時代にも「安心」を保証する

（「安心」は変化するが、やはり重要）

研究の目的：一般化と自動化

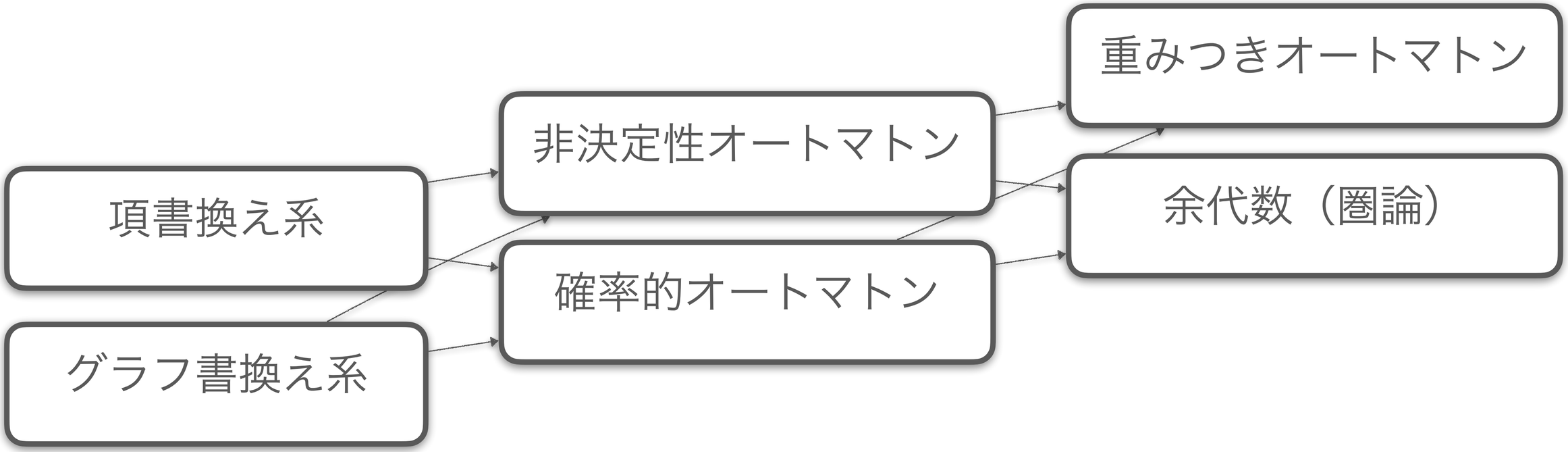
- いつでも「安心」を保証したい！
  - 多様なプログラムをカバーできる一般的な保証がしたい
  - そのためには、一般性のある数学的な表現と証明手法を作る必要がある
    - 難しさ：ある種のプログラムのための表現・証明が、
      別のプログラムには通用しないことがある

- 簡単に「安心」を保証したい！
  - 自動で保証してほしい
  - そのためには、自動化に適した数学的な表現と証明手法を作る必要がある
    - 難しさ：証明での探索範囲が無限になりがち

研究の方法：抽象度の異なる様々な表現を使う

抽象度 **低**
（自動化に向いている）

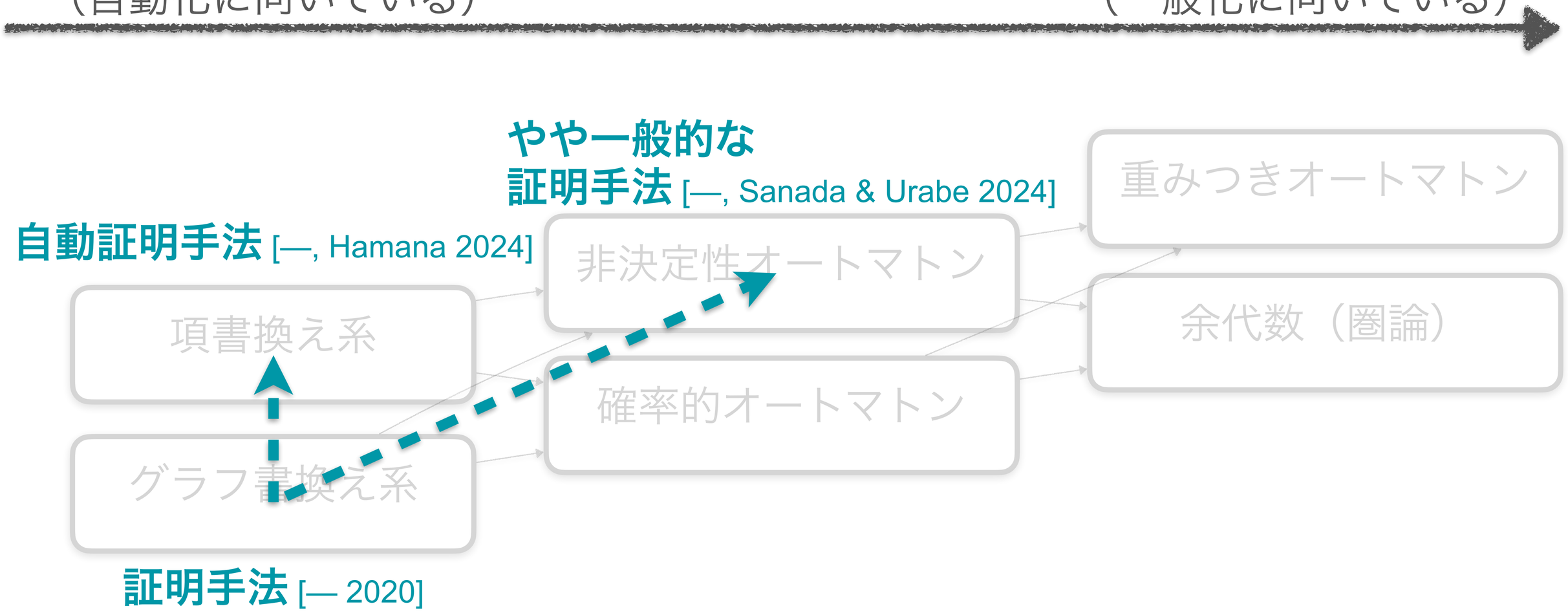
抽象度 **高**
（一般化に向いている）



ここ数年の研究成果：様々な表現を使い証明手法を作る

抽象度 **低**
（自動化に向いている）

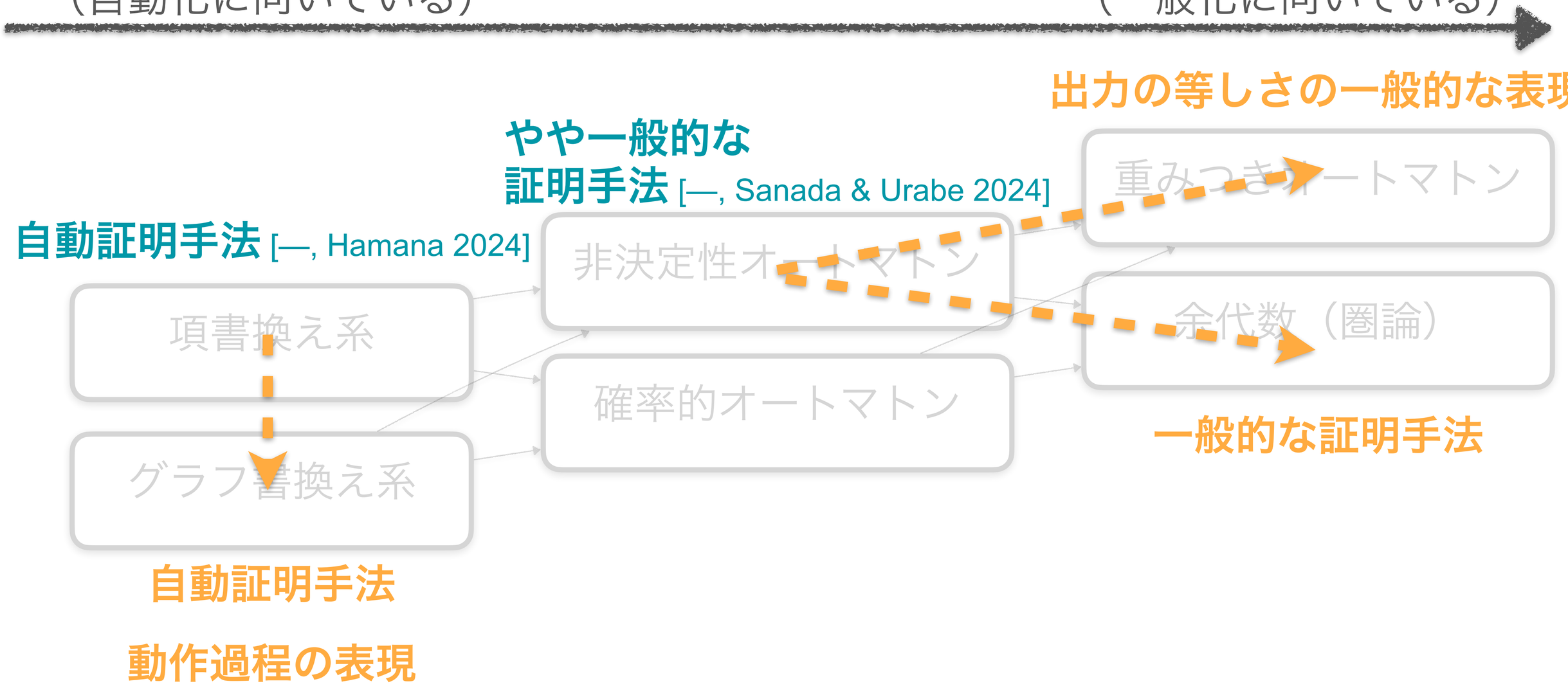
抽象度 **高**
（一般化に向いている）



最近の目論見：自動化と一般性の高みへ

抽象度 **低**
（自動化に向いている）

抽象度 **高**
（一般化に向いている）



いつでも簡単に「安心」を保証する

- 数学的に表現して、証明する
  - 一般化：いつでも、多様なプログラムに
  - 自動化：簡単に、自動で

- そのために、表現と証明を工夫する
  - 抽象度の異なる様々な表現を使う・作る
  - 証明手法を作る・実装する

1. 表現と証明で「安心」を保証する

2. いつでも簡単に「安心」を保証する

（表現と証明の工夫）

3. 簡単から、いつでもへ

（表現と証明の工夫の実例）

4. AI時代にも「安心」を保証する

（「安心」は変化するが、やはり重要）

研究例：自動化から一般化へ

	例1：自動証明手法 [—, Hamana 2024]	例2：やや一般的な証明手法 [—, Sanada & Urabe 2024]
対象のプログラム	出力が一意 違いが局所的	出力が複数
プログラムの 動作過程の表現	項書換え系+α 🆕	非決定性オートマトン
出力の等しさと 実行時間の差の 証明手法	「パターン」の自動判定 🆕	「拡張パターン」の判定 🆕

研究例1：局所性を利用した自動化

	例1：自動証明手法 [—, Hamana 2024]
対象のプログラム	出力が一意 違いが局所的
プログラムの 動作過程の表現	項書換え系+α 🆕
出力の等しさと 実行時間の差の 証明手法	「パターン」の自動判定 🆕

対象の局所性：局所的最適化が生む違い

- 例（”map fusion”）：

  プログラム中の `map f (map g xs)` を

  全て `map (f ∘ g) xs` に置き換える

リストxsを続けて2周して、
関数gと関数fで加工するよりも、
リストxsを1周して、
合成関数f∘gで加工する方が速い

研究例1：局所性を利用した自動化

	例1：自動証明手法 [—, Hamana 2024]
対象のプログラム	出力が一意 違いが局所的
プログラムの 動作過程の表現	項書換え系+α 🆕
出力の等しさと 実行時間の差の 証明手法	「パターン」の自動判定 🆕

表現の局所性：ルールを適用する等式変形

```
   (fun x -> x * x) (1 + 2)
→ (fun x -> x * x) 3
→ 3 * 3
→ 9
```

足し算のルール

代入のルール

掛け算のルール

研究例1：局所性を利用した自動化

	例1：自動証明手法 [—, Hamana 2024]
対象のプログラム	出力が一意 違いが局所的
プログラムの 動作過程の表現	項書換え系+α 🆕
出力の等しさと 実行時間の差の 証明手法	「パターン」の自動判定 🆕

証明の自動化：

1. 局所的な違いと変形ルールの干渉を列挙する

（以下のような P, Q, P'' を列挙する）

プログラム P

→ プログラム P'

プログラム Q

PとQの局所的な違いに
変形ルールが干渉

研究例1：局所性を利用した自動化

	例1：自動証明手法 [—, Hamana 2024]
対象のプログラム	出力が一意 違いが局所的
プログラムの 動作過程の表現	項書換え系+α 🆕
出力の等しさと 実行時間の差の 証明手法	「パターン」の自動判定 🆕

証明の自動化：

2. 「パターン」が完成するか判定する

（以下のような P'', Q' を探索する）

プログラム P

→ プログラム P'

プログラム Q

PとQの局所的な違いに
変形ルールが干渉

→(k回)→
プログラム P''

→(l回)→
プログラム Q'

P''とQ'も局所的に違う
かつ、1+k≥lが成立

研究例1：局所性を利用した自動化

	例1：自動証明手法 [—, Hamana 2024]
対象のプログラム	出力が一意 違いが局所的
プログラムの 動作過程の表現	項書換え系+α 🆕
出力の等しさと 実行時間の差の 証明手法	「パターン」の自動判定 🆕

証明の自動化：

1. 局所的な違いと変形ルールの干渉を列挙する
2. 「パターン」が完成するか判定する

変形ルールは有限個で、自動で列挙・判定が可能！

研究例2：表現の抽象度を上げて一般化へ

対象のプログラムを増やしたい！でも、どこを改良すべき？

	例1：自動証明手法 [—, Hamana 2024]
対象のプログラム	出力が一意 違いが局所的
プログラムの 動作過程の表現	項書換え系+α 🆕
出力の等しさと 実行時間の差の 証明手法	「パターン」の自動判定 🆕

プログラム P
→ プログラム P'
→(k回)→
プログラム P''

プログラム Q
→(l回)→
プログラム Q'

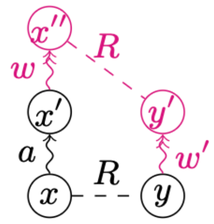
研究例2：表現の抽象度を上げて一般化へ

抽象度を上げると、改良点が浮かび上がる：「パターン」の拡張が必要

	例1：自動証明手法 [—, Hamana 2024]	例1の抽象化
対象のプログラム	出力が一意 違いが局所的	出力が一意
プログラムの 動作過程の表現	項書換え系+α 🆕	非決定性オートマトン
出力の等しさと 実行時間の差の 証明手法	「パターン」の自動判定 🆕	~~「パターン」の判定~~

プログラム P
→ プログラム P'
→(k回)→
プログラム P''

プログラム Q
→(l回)→
プログラム Q'



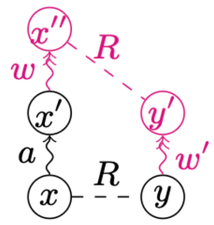
研究例2：表現の抽象度を上げて一般化へ

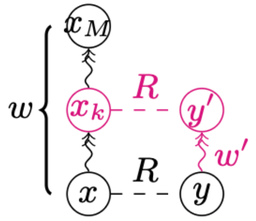
「パターン」の拡張により、対象のプログラムが少し増やせた！

	例1：自動証明手法 [—, Hamana 2024]	例1の抽象化	例2：やや一般的な証明手法 [—, Sanada & Urabe 2024]
対象のプログラム	出力が一意 違いが局所的	出力が一意	出力が複数
プログラムの 動作過程の表現	項書換え系+α NEW	非決定性オートマトン	非決定性オートマトン
出力の等しさと 実行時間の差の 証明手法	「パターン」の自動判定 NEW	~~「パターン」の判定~~	「拡張パターン」の判定 NEW

プログラム P
→ プログラム P'
→(k回)→
プログラム P''

プログラム Q
→(l回)→
プログラム Q'





研究例：簡単から、いつでもへ

	例1：自動証明手法 [—, Hamana 2024]	例1の抽象化	例2：やや一般的な証明手法 [—, Sanada & Urabe 2024]
対象のプログラム	出力が一意 違いが局所的	出力が一意	出力が複数
プログラムの 動作過程の表現	項書換え系+α 🆕	非決定性オートマトン	非決定性オートマトン
出力の等しさと 実行時間の差の 証明手法	「パターン」の自動判定 🆕	~~「パターン」の判定~~	「拡張パターン」の判定 🆕

局所性を利用して
自動化

・・・・→

抽象度を上げて
一般化へ

1. 表現と証明で「安心」を保証する

2. いつでも簡単に「安心」を保証する

（表現と証明の工夫）

3. 簡単から、いつでもへ

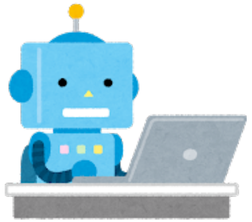
（表現と証明の工夫の実例）

4. AI時代にも「安心」を保証する

（「安心」は変化するが、やはり重要）

AIが変えるもの：プログラムの役割

- プログラムが人とコンピュータ**とAI**をつなぐ
  - 人がAIにプログラムを書かせる



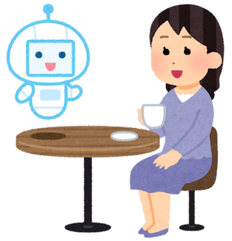
- 「安心」の保証はやはり（むしろ）重要
  - いつでも（一般化）
  - 簡単に（自動化）

AIが変えるもの：プログラムの内容

- プログラムが内製のAIを使う
  - 機械学習モデル（ニューラルネットなど）を作り、学習・推論させる

AIが変えるもの：プログラムの内容

- プログラムが外製のAIを使う
  - LLMに質問して回答を得る



- 「安心」の内容は変化しそう
  - 正しさ：LLMが不確実でも意図通りに動く
  - 速さ
  - お金：LLMを予算以上に使わない
  - 規制：LLMにDoS攻撃をしない

不確実性をはらんだ
正しさとは？

トレードオフがある
「安心」とは？

1. 表現と証明で「安心」を保証する

2. いつでも簡単に「安心」を保証する

（表現と証明の工夫）

3. 簡単から、いつでもへ

（表現と証明の工夫の実例）

4. AI時代にも「安心」を保証する

（「安心」は変化するが、やはり重要）